

@renatonobregadev

Primeiros passos com Spring Boot Actuator

Observabilidade: o que é, como funciona e quando usar.

Por que observabilidade?

Caixa-preta: Sem visibilidade, você só descobre que a aplicação caiu quando o usuário reclama.

Pergunta certa: "Está de pé? Está saudável? Está lenta? Por quê?" Responder isso exige dados vindos de dentro da aplicação.

Na prática: Kubernetes, load balancers e ferramentas de monitoramento precisam de um jeito padrão de perguntar isso ao seu serviço.

O que é o Actuator?

Actuator: Módulo do Spring Boot que expõe informações operacionais da aplicação: saúde, métricas, configurações e logs.

Endpoint: Cada informação vira um endpoint, acessível por HTTP (ou JMX) em `/actuator/{id}`. Ex.: `/actuator/health`.

Pronto para produção: Você não escreve o código de monitoramento do zero. Adiciona uma dependência e ganha dezenas de recursos.

BRASIL JUG

Primeiro passo

Dependência: Adicione **`spring-boot-starter-actuator`** no **`pom.xml`**. Só isso já ativa o módulo.

Padrão seguro: Por padrão, via HTTP, só o **`/actuator/health`** fica exposto. O resto existe, mas está escondido.

Expor mais:

`management.endpoints.web.exposure.include` com **`health,info,metrics`** libera os endpoints que você escolher.



Health check

/actuator/health: Responde se a aplicação está saudável. Ex.: `{"status":"UP"}`.

Status: `UP`, `DOWN`, `OUT_OF_SERVICE` e `UNKNOWN`. Se um componente está `DOWN`, a resposta HTTP vira 503.

Automático: Detectou banco, disco, RabbitMQ ou Redis no projeto? O Actuator já cria um health indicator para cada um.

Detalhando o health

show-details: Por padrão você só vê o status geral. `management.endpoint.health.show-details` aceita `never`, `when-authorized` ou `always`.

Componentes: Com detalhes, aparece o status de cada peça: `db`, `diskSpace`, `rabbit`...

Health customizado: Implemente `HealthIndicator` e retorne `Health.up()` ou `Health.down()` para checar uma API externa, por exemplo.



Liveness vs Readiness

Liveness: "O processo está vivo?" Se falhar, o orquestrador reinicia o container. Em `/actuator/health/liveness`.

Readiness: "Posso receber tráfego?" Se falhar, o pod sai do balanceamento, mas não é reiniciado. Em `/actuator/health/readiness`.

Ativação: Ligados automaticamente no Kubernetes. Fora dele, use `management.endpoint.health.probes.enabled=true`.



Métricas

Micrometer: A "fachada" de métricas do Spring. Você mede uma vez e exporta para Prometheus, Datadog, New Relic...

/actuator/metrics: Lista as métricas disponíveis. Ex.: **`http.server.requests`** mostra contagem e tempo das requisições.

Prometheus: Com **`micrometer-registry-prometheus`**, ganha **`/actuator/prometheus`**, pronto para o Grafana consumir.



Info

/actuator/info: Mostra dados sobre a aplicação: nome, versão, commit, build.

Propriedades: Com `management.info.env.enabled=true`, tudo que começa com `info.` aparece. Ex.: `info.app.versao=1.0.0`.

Git e build: Os plugins `git-commit-id` e `build-info` do Maven adicionam commit e data do build. Ótimo para saber "qual versão está no ar?".



Endpoints úteis no dia a dia

/actuator/loggers: Consulta e altera o nível de log em tempo de execução, via POST. Sem redeploy para ligar o DEBUG.

/env e /configprops: Mostram as propriedades e configurações efetivas. Desde o Boot 3, os valores vêm mascarados por padrão.

/mappings e /beans: Listam todas as rotas HTTP e beans do contexto. Ajudam muito no debug.

Endpoint customizado

@Endpoint(id = "fila"): Cria seu próprio endpoint em `/actuator/fila`.

Operações: `@ReadOperation` (GET), `@WriteOperation` (POST) e `@DeleteOperation` (DELETE).

Quando usar: Para expor algo específico do negócio, como o tamanho de uma fila interna ou um cache que precisa ser limpo.



Segurança

Dados sensíveis: `/heapdump`, `/env` e `/threaddump` podem vazar senhas, tokens e dados de usuários.

Porta separada: `management.server.port=8081` tira o Actuator da porta pública. Só a rede interna acessa.

Spring Security: Proteja `/actuator/**` com autenticação e deixe público só o `health`, se necessário.

Actuator não é uma bala de prata!

Ele expõe, não analisa: O Actuator só fornece os dados. Gráficos, alertas e histórico ficam com Prometheus, Grafana e cia.

Rastreamento: Para seguir uma requisição entre microserviços, você precisa de tracing (Micrometer Tracing + Zipkin, Jaeger ou OpenTelemetry).

O lado bom: É a base da observabilidade em Spring. Barato de adotar e essencial em produção.



Erros comuns

include=* em produção: Expor todos os endpoints publicamente abre a porta para **`heapdump`** e **`env`**. Exponha só o necessário.

Readiness dependendo de tudo: Se a readiness checa uma API externa e ela cai, todos os pods saem do ar juntos. Inclua só o essencial.

show-details=always aberto: Revela versão do banco, espaço em disco e nomes de serviços para qualquer um.



Ficou claro?

- ○ **SALVE** para revisar quando for configurar o monitoramento do seu próximo serviço.
- ○ **COMPARTILHE** com aquele amigo que está colocando a primeira API em produção.
- ○ **COMENTE:** Qual endpoint do Actuator você mais usa? Conta aqui!

BRASIL JUG



SPRING BOOT ACTUATOR

Post criado por: @renatonobregadev